

LOGOS V1 Project Writeup

Logical Optimization for Global Outcome Systems

A quant research system for detecting logical inconsistency in prediction markets

1. Why we are building this

The goal is not to build another generic “quant project.”

We do **not** want to make:

- a basic stock predictor
- a random ML trading bot
- a toy Black-Scholes calculator
- a shallow backtest with no real intellectual spine

Those are crowded, weak, and forgettable.

Instead, we want to build something that shows three things at once:

1. **Mathematical seriousness**
2. **Systems thinking**
3. **Originality of framing**

The core idea behind LOGOS is simple but strong:

Prediction markets claim to represent probabilities.

LOGOS tests whether those probabilities are actually logically possible.

That framing is what makes this project different. We are not trying to predict markets. We are trying to understand whether markets are internally coherent.

This is a stronger intellectual angle because it focuses on **structure**, not hype.

2. Strategic context behind the project

This project came out of a broader effort to build a sharp, precise, innovative deliverable for a serious entry into quant.

A few key ideas shaped the thinking:

A. “The math is the moat.”

A lot of coding work is becoming commoditized. AI can increasingly help write code, prototype models, and automate implementation. That means coding alone is becoming less scarce.

What remains scarce is:

- mathematical fluency
- probabilistic reasoning
- understanding market structure
- optimization
- judgment
- ability to formalize messy systems into logic

So the project needs to show real mathematical and structural thinking, not just programming.

B. Most student quant projects are too common

Most undergrad quant projects fall into very predictable buckets:

- price prediction
- factor backtesting
- momentum/reversion toy strategies
- ML on stock returns

- options pricing demos

Those are fine, but they are crowded. We wanted something less obvious and more conceptually sophisticated.

C. Structure is more impressive than prediction

The most interesting quant work is often not “can I predict the next move?” but:

- what constraints should prices obey?
- how do systems break?
- where does arbitrage come from?
- what relationships are logically necessary?
- how do fragmented markets violate probability laws?

That led naturally to prediction markets and logical inconsistency detection.

3. Why prediction markets, specifically Polymarket

We decided to use **Polymarket** as the V1 market source.

Reasons:

1. **The prices are naturally interpretable as probabilities**
That makes the project conceptually clean.
2. **Prediction markets naturally create logical relationships**
Example:
 - BTC > 95k by date D
 - BTC > 100k by date D
 - BTC > 105k by date D
3. Those contracts cannot be priced independently without implied logical structure.

4. **Polymarket is more aligned with the project idea than equities**
In stocks, coherence relationships are often less explicit. In prediction markets, event logic is much clearer.
5. **It aligns with broader interests in market infrastructure, on-chain systems, and future-facing financial architecture**
This makes the project strategically stronger and more personal.

So V1 will use **Polymarket** as the core data source.

4. The core idea of LOGOS

LOGOS stands for:

Logical Optimization for Global Outcome Systems

We kept this name because it still fits perfectly.

The word “logos” captures:

- logic
- reason
- structure
- order beneath apparent chaos

That is exactly what the system is doing.

Markets often look chaotic on the surface.
LOGOS tries to extract the formal logic underneath them.

The system’s core job is:

1. take observed market probabilities
2. organize related markets into logical families
3. check whether those prices violate basic probability logic

4. compute the nearest coherent probability system
5. measure how inconsistent the market currently is

So LOGOS is best thought of as:

A market logic engine for detecting structural inconsistency in prediction markets.

5. Honest originality assessment

We were careful to be honest about originality.

The underlying idea is **not mathematically new**. The no-arbitrage principle and logical consistency in markets are fundamental ideas in quantitative finance.

So this is **not** some never-before-seen research concept.

However, what makes it strong is this:

- it is uncommon as an undergrad quant project
- it combines probability, optimization, and systems thinking
- it focuses on market coherence rather than naive prediction
- it can be implemented in a way that feels like a research system, not a classroom toy

So the originality is not in inventing a new theorem.

The originality is in the **framing, implementation, and integration**.

That is exactly where good projects become impressive.

6. Why we rejected other project paths

We considered several possible project types.

Option A: generic quant learning engine

This would have been a broad notebook or dashboard showing:

- Bayes
- MLE
- regression
- Black-Scholes
- Monte Carlo
- optimization

That would be academically fine, but too broad and not differentiated enough.

Option B: prediction market mispricing lab

This was stronger. It focused on probability structure and mispricing in event markets.

Option C: full institutional quant stack prototype

This would include ingestion, fair-value engine, signal generation, risk layer, execution simulator, etc.

That is exciting, but too large for a 4-week V1.

So the best path became:

A prediction-market logic engine that detects structural inconsistency and computes the nearest coherent probability system.

That is focused, mathematically serious, and feasible.

7. Why the project is not a prediction system

This point is important.

LOGOS is **not** trying to answer:

- who will win the election
- whether BTC will hit 100k

- which team will win a championship

Instead, it asks:

Given the probabilities currently implied by the market, are those probabilities logically coherent?

That is a very different kind of question.

For example, if the market says:

- $P(\text{BTC} > 95\text{k by June}) = 0.48$
- $P(\text{BTC} > 100\text{k by June}) = 0.52$

then something is wrong, because exceeding 100k is a subset of exceeding 95k. The harder event cannot be more likely than the easier one.

LOGOS exists to detect exactly that kind of inconsistency.

This is stronger than pure prediction because it is about **market structure** and **probability logic**.

8. The mathematical depth we chose

We had to choose the right level of rigor.

Because this is a 4-week V1 and both builders are undergrads, the best choice is:

Undergraduate research level with strong probability and convex optimization

We explicitly decided **not** to center V1 around:

- stochastic calculus
- Itô processes
- Black-Scholes derivations
- measure theory

- full derivatives pricing models

Those are valuable topics, but they are not necessary for the logic-detection engine.

The math that **does** matter for LOGOS is:

1. **Probability theory**

- probabilities as coherent objects
- conditional logic
- subset relationships
- exclusivity and exhaustiveness

2. **Convex optimization**

- projecting observed market probabilities onto the nearest coherent set

3. **Graph / dependency thinking**

- representing related markets as structured families

4. **Empirical statistical analysis**

- frequency of violations
- magnitude of inconsistencies
- persistence over time

That combination is already serious and directly relevant.

9. Timeline choice

We decided on:

4 weeks for LOGOS V1

This is the correct choice.

It is enough time to build a real prototype, but only if the scope is controlled.

We also explicitly treated this as **Version 1**, not the final form.

That means:

- the V1 should be clean, rigorous, and focused
- future versions can add more ambitious features later

Possible future upgrades:

- automatic market relationship extraction with LLMs
- real-time signal generation
- backtesting and trade simulation
- cross-venue arbitrage
- Bayesian updating layer
- richer dependency graphs

But those are V2/V3 ideas.

For now, the right mindset is:

build a strong core, not a sprawling mess.

10. Scope decision: which version fits 4 weeks best?

We considered three scope levels.

Option 1: Detection Engine

This would just flag logical violations.

Example:

- “ $P(\text{win championship}) > P(\text{reach finals})$ ” → violation

Too weak. That alone would not look like serious quant work.

Option 2: Detection + Optimization Engine

This detects violations and computes the nearest coherent probability vector.

This was the winner.

Option 3: Detection + Optimization + Backtest

This would add strategy simulation and profitability testing.

Too risky for 4 weeks. Backtesting prediction markets properly requires:

- high-quality historical data
- timestamps
- fee assumptions
- liquidity assumptions
- execution modeling

That could easily consume the whole timeline.

So LOGOS V1 is:

Detection + Optimization Engine

This is the best tradeoff between rigor and feasibility.

11. The four market family types for LOGOS V1

We finalized four family types that LOGOS will support.

These are the logical templates the engine will use to organize markets.

1. Threshold Chains

Example:

- BTC > 95k by June 30
- BTC > 100k by June 30
- BTC > 105k by June 30

Constraint:

- $P(>95k) \geq P(>100k) \geq P(>105k)$
 $P(>95k) \setminus \geq P(>100k) \setminus \geq P(>105k)$

Why it matters:

- this is the cleanest and strongest family
- easy to explain
- likely to exist in Polymarket
- ideal for V1

This is the **primary family**.

2. Deadline Nesting

Example:

- BTC > 100k by March 31
- BTC > 100k by June 30
- BTC > 100k by December 31

Constraint:

- $P(\text{by March}) \leq P(\text{by June}) \leq P(\text{by December})$
 $P(\text{by March}) \setminus \leq P(\text{by June}) \setminus \leq P(\text{by December})$

Why it matters:

- adds time structure
- very intuitive
- also clean and powerful

This is the **second benchmark family**.

3. Nested Achievement Chains

Example:

- Team makes playoffs
- Team reaches conference final
- Team wins championship

Constraint:

- $P(\text{championship}) \leq P(\text{conference final}) \leq P(\text{playoffs})$
 $P(\text{championship}) \leq P(\text{conference final}) \leq P(\text{playoffs})$

Why it matters:

- shows the engine can reason beyond numeric thresholds
- introduces more realistic event dependency logic

This family requires more manual curation, but it makes the project feel more general and sophisticated.

4. Mutually Exclusive Outcome Sets

Example:

- Candidate A wins

- Candidate B wins
- Candidate C wins

Constraint:

- if exhaustive: probabilities sum to 1
- if only mutually exclusive: probabilities sum to at most 1

Why it matters:

- broadens the logic from monotonicity to exclusivity/coherence
- adds another type of constraint family

This one must be used carefully because the market set must be clearly exhaustive or clearly not.

12. What we deliberately excluded from V1

We consciously rejected several tempting additions.

A. Cross-venue arbitrage

Interesting, but too messy and too infrastructure-heavy.

B. Full backtesting

Too risky for a 4-week V1.

C. LLM-based dependency extraction

Exciting, but not necessary for the first version.

D. Resolution timing / mempool / front-running logic

Interesting for future work, but not the right first build.

The principle here was:
finish a strong core before adding complexity.

13. The key design decision for relationship discovery

One of the most important choices was how LOGOS should discover market relationships.

We considered using an LLM to automatically infer relationships from market descriptions.

That was rejected for V1.

Instead, we chose a **hybrid relationship builder**:

- rule-based grouping for clean families
- manual curation for more complex families
- structured family templates

This is the right decision because:

- it is more reliable
- it is easier to defend mathematically
- it reduces project fragility
- it keeps the focus on logic and optimization, not noisy NLP

So V1 will not attempt to understand “the whole market.”

It will instead build **high-confidence logic families**.

That is more rigorous.

14. The optimization formulation

This is the mathematical heart of LOGOS.

For each family, we observe a market probability vector:

$p_{\text{market}} p^{\{\text{market}\}} p_{\text{market}}$

LOGOS then computes the nearest coherent probability vector:

$p^* p^* p^*$

by solving:

$$\min_p \sum_i w_i (p_i - p_{\text{market}})^2 \quad \min_{\{p\}} \sum_i w_i (p_i - p_{\text{market}})^2$$

subject to:

- family-specific logical constraints
- $0 \leq p_i \leq 1$

The initial plan is to use equal weights:

$w_i = 1$

A possible refinement later is to weight by liquidity or volume so that highly liquid contracts move less in the projection.

The reason this optimization is so strong conceptually is that LOGOS is not just saying:

“this market is wrong”

It is saying:

“here is the closest logically coherent probability system consistent with the observed market.”

That is much more elegant and much more quant-like.

15. Example constraints

Here is what the engine will enforce for different families.

Threshold Chain

$p_{95} \geq p_{100} \geq p_{105} \geq p_{95}$

Deadline Nesting

$$p_{\text{March}} \leq p_{\text{June}} \leq p_{\text{December}} \quad p_{\text{March}} \leq p_{\text{June}} \leq p_{\text{December}}$$

Nested Achievement

$$p_{\text{championship}} \leq p_{\text{finals}} \leq p_{\text{playoffs}} \quad p_{\text{championship}} \leq p_{\text{finals}} \leq p_{\text{playoffs}}$$

Mutually Exclusive Set

$$\sum_i p_i = 1$$

or

$$\sum_i p_i \leq 1$$

depending on the family.

All of these are linear constraints, which makes the optimization clean and tractable.

16. What LOGOS should output

For each market family, LOGOS should produce:

1. **Observed market probabilities**
2. **Coherent projected probabilities**
3. **Per-market adjustment / mispricing**
4. **A family-level inconsistency score**

A natural family-level inconsistency score is:

$$\frac{\|p_{\text{market}} - p^*\|_2}{\|p^*\|_2}$$

This tells us how far the observed market system is from logical coherence.

That score becomes a central metric in the analysis.

17. The three main empirical questions

To make the project feel like research rather than just a tool, we centered it on three empirical questions.

A. Frequency

How often are market families logically inconsistent?

Metrics:

- fraction of snapshots with violations
- fraction of families with violations
- violations by family type

This tells us whether inconsistency is rare or common.

B. Magnitude

How large are the inconsistencies?

Metrics:

- average adjustment size
- max per-market deviation
- family inconsistency score
- possibly volume-weighted inconsistency

This distinguishes tiny noise from meaningful structural breaks.

C. Persistence

How long do violations last before they disappear?

Metrics:

- average duration
- median duration

- persistence by family type
- persistence by liquidity bucket

This matters because if violations last long enough, they become more relevant from an arbitrage and market efficiency perspective.

18. The final conceptual architecture

The project architecture we settled on is:

1. **Polymarket API / data source**
2. **Data ingestion layer**
3. **Market parser**
4. **Family builder**
5. **Constraint engine**
6. **Convex optimization engine**
7. **Metrics layer**
8. **Visualization + research output layer**

In plain English:

- fetch market data
- structure it
- group related contracts
- impose logic
- compute coherent probabilities

- measure inconsistency
- present results cleanly

That is the full V1 system.

19. Repo / implementation architecture

The intended repo organization is modular.

Core modules:

- ingestion
- parsing
- family construction
- constraints
- optimization
- metrics
- visualization

The minimum viable files to make the engine real are:

1. `schema.py`
2. `polymarket_client.py`
3. `title_parser.py`
4. `builder.py`
5. `base.py` for constraints

6. `projector.py`

7. `inconsistency.py`

These are the pieces that matter most.

20. What matters most in execution

We were very clear that the most important parts of LOGOS are **not**:

- branding
- fancy dashboarding
- exaggerated ambition

The most important parts are:

1. **Good market family construction**
2. **Correct logical constraints**
3. **Clean optimization outputs**
4. **Real empirical findings**

If those four things are strong, the project is strong.

21. Division of work

The working division of labor we discussed is:

Caleb

- project framing

- Polymarket data ingestion
- family design and curation
- visualization
- interpretation and writing
- presentation / positioning

Northwestern collaborator

- formal optimization formulation
- constraint implementation
- cvxpy projection layer
- numerical analysis

Shared

- debugging
- family detection logic
- empirical analysis
- testing
- final polishing

This split makes sense given the strengths involved.

22. Why this project is worth building

LOGOS is worth building because it demonstrates a rare combination of abilities:

- probabilistic reasoning
- formalization of market logic
- optimization
- empirical market analysis
- system design
- disciplined scoping

Most importantly, it shows a mindset that is more interesting than “I want to make a trading bot.”

It shows the ability to ask:

What assumptions are built into prices?

Are those assumptions coherent?

When they are not, how can we formalize the break?

That is a stronger intellectual signal.

23. The final V1 statement

The cleanest way to describe the project is:

LOGOS (Logical Optimization for Global Outcome Systems) is a quant research system that detects logical inconsistencies in curated Polymarket market families and computes the nearest coherent probability system using convex optimization.

Or even more simply:

LOGOS tests whether prediction markets obey their own implied probability logic.

That is the project.

24. V1 success criteria

LOGOS V1 is successful if it can show that:

1. real prediction market families sometimes violate logical probability constraints
2. those violations can be formally detected
3. a coherent probability system can be computed via optimization
4. the frequency, magnitude, and persistence of violations can be measured empirically

That is enough for a strong first version.

25. Future extensions

If V1 goes well, possible next iterations include:

- automated dependency extraction with LLMs
- real-time detection dashboard
- backtesting and trade simulation
- cross-venue consistency checks
- richer Bayesian updating layers
- on-chain event graph enrichment

But those are later.

The priority now is:

Build a clean, rigorous V1 that actually works.